UNIT V: Virtual Functions, Polymorphism & Exception Handling

Question Bank: Debugging & Pseudo-Code Formulation (Questions Only)

---

1. Syllabus & Topic Scope

- **Virtual Functions**: Purpose and need for virtual functions, static/early binding vs. dynamic/late binding, member function dynamic dispatch.
- **Internal Mechanics of Virtual Functions**: Virtual method table (`vtable`), virtual table pointer (`vptr`), compiler overhead and indirection cost.
- **Virtual Destructors**: Why base classes with virtual functions require virtual destructors, preventing undefined behavior and memory leaks during polymorphic deletion via base pointers.
- **Pure Virtual Functions**: Syntax (`virtual void func() = 0;`), semantic role in defining pure abstract interfaces.
- **Runtime Polymorphism**: Achieving uniform runtime interfaces using base class pointers and references pointing to derived class instances.
- **Exception Handling Mechanism**: Philosophy of structured error handling, `try`, `catch`, and `throw` keywords, control flow transfer during exceptions.
- **Stack Unwinding**: Automatic destruction of local stack objects during exception propagation.
- **Throwing, Catching & Rethrowing Exceptions**: Throwing primitive values, standard library exceptions, and custom exception classes; catching by reference to prevent object slicing; rethrowing exceptions using bare `throw;` vs. copy-slicing `throw e;`.
- **Catch-All Handler (`catch(...)`)**: Universal catch block, placement as the final fallback.
- **Order of Catch Handlers**: Subclass-before-superclass catch ordering requirement to prevent unreachable handlers.
- **Specifying Exceptions**: Dynamic exception specifications (historical context) and modern `noexcept` specifiers.

---

2. Assessment Instructions & Rules ("How to Do & What to Do")

A. Instructions for Debugging Questions

When debugging runtime polymorphism and exception handling in C++:

1. **Verify Virtual Destructor Presence**:
   - Inspect base classes designed for polymorphic deletion (`delete basePtr;`). If the base class destructor is not declared `virtual`, the derived class destructor will NOT execute, resulting in memory leaks and Undefined Behavior.
2. **Inspect Catch Handler Ordering**:

- Check the order of `catch` blocks. In C++, catch handlers match sequentially. If a base class exception handler (`catch (const BaseException &e)`) precedes a derived class handler (`catch (const DerivedException &e)`), the derived handler is unreachable.

3. Examine Exception Rethrowing:

- Inspect catch blocks that rethrow exceptions. Writing `throw e;` creates a copy with the static type of `e`, causing **Exception Slicing**. To preserve the dynamic derived exception type, bare `throw;` must be used.

4. Identify Abstract Class Instantiation:

- Check whether a derived class inherits pure virtual functions (`= 0`). If the derived class fails to provide an implementation for even a single pure virtual function, it remains an abstract class and cannot be instantiated directly.

5. Inspect Polymorphic Parameter Passing:

- Verify whether functions accepting polymorphic objects take them **by reference** (`const Base&`) or **by pointer** (`Base*`). Passing by value (`Base b`) triggers object slicing and suppresses dynamic binding via `vtable`.

6. Required Output Format:

- State line numbers and classify the bug.
- Provide the theoretical analysis referencing `vtable`/`vptr` mechanics or exception handling runtime semantics.
- Provide the corrected C++ code.
- Provide the dry run and terminal output.

B. Instructions for Pseudo-Code Writing Questions

1. Polymorphic Architecture Modeling:

- Define abstract interfaces using `ABSTRACT CLASS` and `PURE VIRTUAL METHOD method()`.
- Model derived overrides with `OVERRIDE METHOD method()`.
- Include virtual destructors: `VIRTUAL DESTRUCTOR()`.

2. Exception Flow Modeling:

- Use standard exception handling syntax: `TRY`, `THROW NEW ExceptionType(...)`, `CATCH (e AS ExceptionType)`, `RETHROW`, `CATCH (...)`.

3. Traceability:

- Include a trace table tracking control jumps during normal execution, exception throw events, stack unwinding destruction, and catch handler execution.

---

3. Section A: Debugging Questions (5 Problems)

Question 1: Non-Virtual Destructor Memory Leak in Polymorphic Deletion

Topic: Virtual Functions, Virtual Destructors, Dynamic Dispatch

Problem Statement

A graphical rendering engine manages polygon objects through base class `Shape*` pointers. When objects are deleted through the base pointer, the derived destructor fails to run, causing severe memory leaks of dynamically allocated vertex coordinate arrays.

Buggy Code

```
#include <iostream>

class Shape {
public:
    Shape() { std::cout << "Shape created." << std::endl; }

    // Line 7: Non-virtual destructor in base class!
    ~Shape() {
        std::cout << "Shape destroyed." << std::endl;
    }

    virtual void render() const {
        std::cout << "Rendering generic shape." << std::endl;
    }
};

class ComplexPolygon : public Shape {
private:
    double *vertexArray;
    int vertexCount;

public:
    ComplexPolygon(int count) : vertexCount(count) {
        vertexArray = new double[vertexCount * 2];
        std::cout << "Allocated " << vertexCount << " vertices on heap." << std::endl;
    }

    ~ComplexPolygon() {
        std::cout << "Deallocating polygon vertex heap memory." << std::endl;
        delete[] vertexArray; // Never executed!
    }

    void render() const override {
        std::cout << "Rendering polygon with " << vertexCount << " vertices." <<
std::endl;
    }
};

int main() {
    Shape *shapePtr = new ComplexPolygon(1000);
    shapePtr->render();

    // Line 41: Polymorphic deletion through base pointer
    delete shapePtr;

    return 0;
}
```

Student Tasks

1. Explain why `~ComplexPolygon()` is never called when `delete shapePtr;` executes on **Line 41**. How does the compiler resolve destructor calls in the absence of the `virtual` keyword?

2. State the C++ standard rule regarding deleting a derived object through a base class pointer that lacks a virtual destructor. What kind of behavior is this classified as?

3. Add the `virtual` keyword in the appropriate location and explain how the virtual method table (`vtable`) ensures proper destructor chaining from derived to base.

4. Show the complete corrected code and its terminal output.

Question 2: Inverted Exception Catch Handlers & Unreachable Code

Topic: Exception Handling Mechanism, Catch Handler Ordering

Problem Statement

A banking software application defines a hierarchy of custom exceptions. A developer writes a `try-catch` block to handle withdrawals, but the specific derived exception handler for overdrafts is never reached because the base exception handler shadows it.

Buggy Code

```
#include <iostream>
#include <string>

// Base custom exception
class BankingException {
protected:
    std::string msg;
public:
    BankingException(std::string m) : msg(m) {}
    virtual std::string getMessage() const { return msg; }
};

// Derived specific exception
class InsufficientFundsException : public BankingException {
private:
    double shortfall;
public:
    InsufficientFundsException(std::string m, double sf)
        : BankingException(m), shortfall(sf) {}

    double getShortfall() const { return shortfall; }
};

void processWithdrawal(double balance, double amount) {
    if (amount > balance) {
        throw InsufficientFundsException("Account overdrawn!", amount - balance);
    }
}

int main() {
    try {
        processWithdrawal(500.0, 750.0);
    }
    // Line 34: Base handler placed FIRST!
    catch (const BankingException &be) {
        std::cout << "General Banking Error: " << be.getMessage() << std::endl;
    }
    // Line 38: Specific derived handler placed SECOND!
```

```
    catch (const InsufficientFundsException &ife) {
        std::cout << "Shortfall of: $" << ife.getShortfall() << std::endl;
    }

    return 0;
}
```

Student Tasks

1. Identify the warning/error generated by compilers on **Line 38** (`exception of type 'InsufficientFundsException' will be caught by earlier handler`).

2. Explain the mechanism of catch-matching in C++. Why does `catch (const BankingException &be)` match an object of type `InsufficientFundsException`?

3. State the rule for ordering exception handlers in C++ when dealing with inheritance hierarchies.

4. Provide the corrected `try-catch` block including a catch-all fallback handler (`catch(...)`).

Question 3: Exception Rethrowing Slicing Bug (`throw e;` vs. `throw;`)

Topic: Throwing, Catching & Rethrowing an Exception, Object Slicing

Problem Statement

An intermediate error-logging routine intercepts database errors to log audit events and rethrows them for client handling in `main()`. However, the client fails to catch the specific derived `TimeoutError` because the intermediate routine accidentally slices the exception during rethrow.

Buggy Code

```
#include <iostream>
#include <exception>
#include <string>

class DatabaseError : public std::exception {
protected:
    std::string errorDetails;
public:
    DatabaseError(std::string msg) : errorDetails(msg) {}
    const char* what() const noexcept override {
        return errorDetails.c_str();
    }
};

class TimeoutError : public DatabaseError {
public:
    int timeoutMs;
    TimeoutError(std::string msg, int ms)
        : DatabaseError(msg), timeoutMs(ms) {}
};

void executeQuery() {
    throw TimeoutError("Query timed out connecting to shard 3", 5000);
}
```

```
void intermediateAuditor() {
    try {
        executeQuery();
    }
    catch (const DatabaseError &e) {
        std::cout << "[AUDIT LOG]: Intercepted DB error." << std::endl;

        // Line 32: Slicing rethrow bug!
        throw e; // Throws a copy of static type DatabaseError!
    }
}

int main() {
    try {
        intermediateAuditor();
    }
    catch (const TimeoutError &te) {
        std::cout << "Client caught TimeoutError! Duration: " << te.timeoutMs << " ms" <<
std::endl;
    }
    catch (const DatabaseError &de) {
        std::cout << "Client caught generic DatabaseError: " << de.what() << std::endl;
    }
    return 0;
}
```

Student Tasks

1. Explain what output the program currently prints in `main()` and why `catch (const TimeoutError &te)` is bypassed.

2. What is the fundamental difference between writing `throw e;` and bare `throw;` inside a catch block? How does copy construction interact with `throw e;`?

3. Correct **Line 32** to ensure the original polymorphic exception object propagates without loss of derived attributes.

4. Show the corrected output.

Question 4: Attempting to Instantiate an Incomplete Abstract Class

Topic: Pure Virtual Functions, Abstract Classes, Runtime Interfaces

Problem Statement

A networking library defines an abstract interface `NetworkPacket` with pure virtual functions. A derived class `VideoPacket` attempts to create an object, but the compiler aborts compilation because the derived class omitted the implementation of one of the pure virtual functions.

Buggy Code

```
#include <iostream>

class NetworkPacket {
public:
    virtual ~NetworkPacket() {}
```

```
    // Pure Virtual Functions
    virtual void serialize() = 0;
    virtual int getPacketSize() = 0;
};

// Subclass fails to override getPacketSize()
class VideoPacket : public NetworkPacket {
private:
    int frameID;

public:
    VideoPacket(int id) : frameID(id) {}

    void serialize() override {
        std::cout << "Serializing H.264 Video Frame: " << frameID << std::endl;
    }
    // Missing getPacketSize() override!
};

int main() {
    // Line 25: Attempting to instantiate VideoPacket!
    VideoPacket packet(1042);
    packet.serialize();
    return 0;
}
```

Student Tasks

1. State the exact compilation error that occurs on **Line 25**.

2. What technical condition makes a class an "Abstract Class" in C++? Can an abstract class ever be directly instantiated?

3. Under what conditions does a class derived from an abstract base class become a concrete, instantiable class?

4. Provide the corrected C++ code implementing all pure virtual functions.

Question 5: Pass-by-Value Object Slicing Suppressing Dynamic Binding

Topic: Runtime Polymorphism, Early vs. Late Binding, Object Slicing

Problem Statement

A battle simulator game uses runtime polymorphism to handle creature encounters. When entities are passed into a dispatch function `triggerEncounter`, the derived creature behavior (`Dragon::attack`) is suppressed, and generic base class behavior (`Entity::attack`) executes instead.

Buggy Code

```
#include <iostream>

class Entity {
public:
    virtual ~Entity() {}
```

```
    virtual void attack() const {
        std::cout << "Entity performs generic scratch." << std::endl;
    }
};

class Dragon : public Entity {
public:
    void attack() const override {
        std::cout << "Dragon breathes scorching fire!" << std::endl;
    }
};

// Line 17: Takes polymorphic parameter by value!
void triggerEncounter(Entity e) {
    e.attack(); // Calls Entity::attack(), not Dragon::attack()
}

int main() {
    Dragon boss;

    // Developer expects "Dragon breathes scorching fire!"
    triggerEncounter(boss);

    return 0;
}
```

Student Tasks

1. Explain why `triggerEncounter(boss)` prints `"Entity performs generic scratch."`.

2. Explain the role of the virtual table pointer (`vptr`) during pass-by-value versus pass-by-reference. What happens to the `vptr` inside the copied `Entity e` object?

3. Refactor `triggerEncounter` to enable late binding / dynamic dispatch.

4. Show the verified terminal output.

---

4. Section B: Pseudo-Code Writing Questions (5 Problems)

Question 1: Universal Payment Gateway with Runtime Polymorphism

Topic: Pure Virtual Functions, Runtime Polymorphism, Abstract Base Classes

Scenario

Design an enterprise payment transaction dispatcher using **Pure Virtual Functions & Runtime Polymorphism**:

- Abstract Base Class: `PaymentGateway`
  - Pure Virtual Method: `Authorize(amount AS REAL) RETURNS BOOLEAN`
  - Pure Virtual Method: `Capture() RETURNS BOOLEAN`
  - Virtual Destructor
- Concrete Subclasses:

- `CreditCardGateway`: Validates card length (16 digits) and 3-digit CVV.
- `CryptoGateway`: Verifies wallet hash prefix (`"0x"`) and simulates gas fee validation.

- In `Main`, dynamically instantiate the chosen gateway via a base pointer `PaymentGateway*`, authorize and capture funds, and safely deallocate the instance.

Functional Requirements

1. Delineate the abstract base class and pure virtual method signatures.

2. Implement overriding methods in `CreditCardGateway` and `CryptoGateway`.

3. In `Main`, demonstrate polymorphic late binding where the method call `gatewayPtr->Authorize(1500.00)` executes the derived class implementation based on runtime pointer assignment.

Student Tasks

1. Write the formal structured pseudo-code for the payment gateway architecture.

2. Trace the execution flow for a \$1,500 cryptocurrency payment, showing each method dispatched through the base pointer.

Question 2: Resilient Banking Engine with Nested Try-Catch & Rethrowing

Topic: Custom Exception Hierarchy, Nested `try-catch`, Rethrowing

Scenario

Design a fault-tolerant banking transaction engine that implements **Custom Exception Hierarchies**, **Nested Try-Catch**, and **Exception Rethrowing**:

- Custom Exception Classes:
  - `TransactionException` (Base class with attribute `reason AS STRING`).
  - `AccountFrozenException` (Inherits `TransactionException`).
  - `DailyLimitExceededException` (Inherits `TransactionException`, adds `excessAmount AS REAL`).
- Processing Layers:
  - Low-level function `DebitLedger()`: Throws exceptions if account is frozen or daily limit ($50,000) is exceeded.
  - Intermediate function `AuditTransactionWrapper()`: Intercepts `TransactionException`, writes a security audit log, and **rethrows** (`RETHROW`) the active exception.
  - High-level `Main()`: Catches specific derived exceptions and displays appropriate client notices.

Functional Requirements

1. Define the exception inheritance hierarchy.

2. Structure the nested `TRY-CATCH` blocks across the two calling tiers.

3. Explicitly demonstrate the `RETHROW` operation in the middle tier.

Student Tasks

1. Write the structured pseudo-code for the classes and functions.

2. Construct a dry-run trace table tracking control jumps when a withdrawal of \$1,000 is attempted on a frozen account.

Question 3: Polymorphic Game Engine with Virtual Destructors

Topic: Virtual Functions, Virtual Destructors, Polymorphic Object Lifecycles

Scenario

Design a game entity lifecycle manager that processes game objects polymorphically and ensures leak-free destruction:

- Abstract Base Class `GameObject`:
  - Pure virtual methods: `Update(deltaTime AS REAL)` and `Render()`.
  - Virtual destructor `~GameObject()`.
- Derived Class `ParticleSystem`:
  - Dynamically allocates an array of particle coordinates in heap memory.
  - Overrides `Update`, `Render`, and the destructor (releasing dynamic particle array).
- Driver Program:
  - Allocate game entities dynamically and store in an array of base pointers `GameObject*`.
  - Execute a simulation step (`Update` and `Render`).
  - Deallocate all entities using `DEALLOCATE sceneEntities[i]`, relying on virtual destructors to prevent heap leaks.

Student Tasks

1. Write the formal structured pseudo-code showing base and derived classes with virtual destructors.

2. Trace the cleanup phase, explaining the exact destructor execution order when `DEALLOCATE` is called on a base pointer.

Question 4: File Stream Pipeline with Stack Unwinding & Catch-All

Topic: Stack Unwinding, Resource Acquisition (RAII), Catch-All Handler

Scenario

Develop the pseudo-code for a mission-critical File Ingestion Pipeline that demonstrates **Stack Unwinding**:

- Class `ResourceGuard`: Manages a hardware/file descriptor handle. Constructor acquires handle; destructor automatically releases handle.
- Custom Exception: `CRCValidationException` (contains expected and calculated checksums).
- Function `IngestDataStream(fileName)`:
  - Creates a local automatic instance of `ResourceGuard`.
  - Reads data blocks and computes CRC checksum.
  - If CRC mismatches, throws `CRCValidationException`.
- Driver `Main()`:
  - Encloses call in `TRY-CATCH`.
  - Demonstrates that during exception throw, `ResourceGuard`'s destructor is automatically invoked during **Stack Unwinding** before the catch block executes.
  - Employs both specific catch and universal catch-all `CATCH (...)`.

Student Tasks

1. Write the structured pseudo-code demonstrating RAII and stack unwinding.

2. Draw or trace the sequence of events showing how the local stack object is destroyed when a CRC error triggers an exception.

Question 5: Scientific Telemetry Calibration with `noexcept` Contracts

Topic: Specifying Exceptions, `noexcept` Contracts, Boundary Assertions

Scenario

Develop an aerospace telemetry validator demonstrating **Exception Specifications / `noexcept` Contracts**:

- Function `CalibrateThermalSensors(rawVoltage AS REAL) RETURNS REAL`:
  - Potentially throwing function (`noexcept(FALSE)`).
  - Computes temperature from raw voltage.
  - Validates bounds (-273.15°C to $+1500.0^\circ\text{C}$). Throws `OutOfBoundsException` if violated.
- Function `CommitTelemetry(finalTemp AS REAL) NOEXCEPT`:
  - Strictly non-throwing function (`NOEXCEPT` contract).
  - Commits reading to avionics hardware bus.
  - Internally catches and suppresses all exceptions to guarantee the `noexcept` contract is never broken (preventing system abort).
- Driver `Main()` orchestrates calibration and commitment within a structured safety harness.

Student Tasks

1. Write the structured pseudo-code showing explicit throwing rules and `noexcept` enforcement.

2. Trace the pipeline with a faulty voltage reading producing -300.0°C, showing the exception caught in `Main`.

SECTION A – DEBUGGING QUESTIONS

Question 1: Non-Virtual Destructor Memory Leak in Polymorphic Deletion

- Line 7 – Defect category: missing virtual destructor in a polymorphic base class.
- When delete shapePtr is executed, shapePtr has static type Shape* but points to a ComplexPolygon. Deleting a derived object through a base pointer when the base destructor is non-virtual has undefined behavior. It is not merely a guaranteed case where only the base destructor runs.
- Correction: declare ~Shape() virtual. A virtual destructor participates in dynamic dispatch, so destruction begins with ComplexPolygon and then continues to Shape. The derived destructor releases vertexArray.

```
#include <iostream>

class Shape {
public:
    Shape() {
        std::cout << "Shape created." << std::endl;
    }

    virtual ~Shape() {
        std::cout << "Shape destroyed." << std::endl;
    }

    virtual void render() const {
        std::cout << "Rendering generic shape." << std::endl;
    }
};

class ComplexPolygon : public Shape {
private:
    double* vertexArray;
    int vertexCount;

public:
    ComplexPolygon(int count) : vertexCount(count) {
        vertexArray = new double[vertexCount * 2];
        std::cout << "Allocated " << vertexCount
                  << " vertices on heap." << std::endl;
    }

    ~ComplexPolygon() override {
        std::cout << "Deallocating polygon vertex heap memory."
                  << std::endl;
        delete[] vertexArray;
    }

    void render() const override {
        std::cout << "Rendering polygon with "
                  << vertexCount << " vertices." << std::endl;
    }
};

int main() {
    Shape* shapePtr = new ComplexPolygon(1000);
    shapePtr->render();
```

```
    delete shapePtr;
    return 0;
}
```

```
Shape created.
Allocated 1000 vertices on heap.
Rendering polygon with 1000 vertices.
Deallocating polygon vertex heap memory.
Shape destroyed.
```

Destruction order: ComplexPolygon::~ComplexPolygon() executes first, releasing the derived resource; then Shape::~Shape() executes. The compiler typically implements virtual dispatch using a vtable and a vptr, but their exact layout is implementation-dependent.

Question 2: Inverted Exception Catch Handlers and Unreachable Code

- Line 38 – Defect category: catch-handler ordering. A compiler commonly diagnoses the derived handler as unreachable because the earlier BankingException handler can already catch InsufficientFundsException.
- A derived exception object can bind to a reference to its public base class. Therefore catch (const BankingException&) matches an InsufficientFundsException too.
- Rule: place handlers from the most specific derived exception type to the more general base type. A catch(...) handler must be last because it catches any exception not matched earlier.

```
#include <iostream>
#include <string>

class BankingException {
protected:
    std::string msg;

public:
    BankingException(std::string m) : msg(m) {}
    virtual std::string getMessage() const { return msg; }
    virtual ~BankingException() = default;
};

class InsufficientFundsException : public BankingException {
private:
    double shortfall;

public:
    InsufficientFundsException(std::string m, double sf)
        : BankingException(m), shortfall(sf) {}

    double getShortfall() const { return shortfall; }
};

void processWithdrawal(double balance, double amount) {
    if (amount > balance) {
        throw InsufficientFundsException(
            "Account overdrawn!", amount - balance);
    }
}

int main() {
```

```
    try {
        processWithdrawal(500.0, 750.0);
    }
    catch (const InsufficientFundsException& ife) {
        std::cout << "Shortfall of: $" << ife.getShortfall()
                  << std::endl;
    }
    catch (const BankingException& be) {
        std::cout << "General Banking Error: "
                  << be.getMessage() << std::endl;
    }
    catch (...) {
        std::cout << "Unknown banking error." << std::endl;
    }
    return 0;
}
```

```
Shortfall of: $250
```

Question 3: Exception Rethrowing Slicing Bug – throw e versus throw

- Line 32 – Defect category: exception slicing during rethrow.
- With throw e;, a new exception object is thrown from the expression e, whose static type is DatabaseError. The derived TimeoutError-specific member timeoutMs is lost. Consequently, main() does not match catch (const TimeoutError&) and instead enters the DatabaseError handler.
- Bare throw; rethrows the currently handled exception object without creating a sliced base-class copy. It preserves the original dynamic type and its derived data.

```
#include <iostream>
#include <exception>
#include <string>

class DatabaseError : public std::exception {
protected:
    std::string errorDetails;

public:
    DatabaseError(std::string msg) : errorDetails(msg) {}

    const char* what() const noexcept override {
        return errorDetails.c_str();
    }
};

class TimeoutError : public DatabaseError {
public:
    int timeoutMs;

    TimeoutError(std::string msg, int ms)
        : DatabaseError(msg), timeoutMs(ms) {}
};

void executeQuery() {
    throw TimeoutError(
        "Query timed out connecting to shard 3", 5000);
}
```

```
void intermediateAuditor() {
    try {
        executeQuery();
    }
    catch (const DatabaseError& e) {
        std::cout << "[AUDIT LOG]: Intercepted DB error."
                  << std::endl;
        throw; // Rethrow the original exception.
    }
}

int main() {
    try {
        intermediateAuditor();
    }
    catch (const TimeoutError& te) {
        std::cout << "Client caught TimeoutError! Duration: "
                  << te.timeoutMs << " ms" << std::endl;
    }
    catch (const DatabaseError& de) {
        std::cout << "Client caught generic DatabaseError: "
                  << de.what() << std::endl;
    }
    return 0;
}
```

Expected behavior of the original buggy version: the audit log is printed, followed by the generic DatabaseError message; the TimeoutError handler is bypassed because throw e creates a DatabaseError exception object.

```
[AUDIT LOG]: Intercepted DB error.
Client caught TimeoutError! Duration: 5000 ms
```

Question 4: Attempting to Instantiate an Incomplete Abstract Class

- Line 25 – Compilation error: VideoPacket is abstract and cannot be instantiated because it has not overridden the pure virtual function getPacketSize().
- A class is abstract if it has at least one pure virtual function for which no final overrider is provided. An abstract class cannot be directly instantiated.
- A derived class becomes concrete only after it implements every inherited pure virtual function with a compatible override.

```
#include <iostream>

class NetworkPacket {
public:
    virtual ~NetworkPacket() = default;
    virtual void serialize() = 0;
    virtual int getPacketSize() = 0;
};

class VideoPacket : public NetworkPacket {
private:
    int frameID;

public:
    VideoPacket(int id) : frameID(id) {}

    void serialize() override {
        std::cout << "Serializing H.264 Video Frame: "
                  << frameID << std::endl;
    }

    int getPacketSize() override {
        return 1024; // Illustrative encoded packet size in bytes.
    }
};

int main() {
    VideoPacket packet(1042);
    packet.serialize();
    std::cout << "Packet size: "
              << packet.getPacketSize() << " bytes" << std::endl;
    return 0;
}
```

```
Serializing H.264 Video Frame: 1042
Packet size: 1024 bytes
```

The question bank does not specify a real encoding-size calculation. The value 1024 bytes is an illustrative return value used to demonstrate a valid override.

Question 5: Pass-by-Value Object Slicing Suppressing Dynamic Binding

- Line 17 – Defect category: object slicing caused by pass-by-value polymorphic parameter.
- triggerEncounter(Entity e) constructs a separate Entity parameter from the Entity base subobject of boss. The Dragon-specific part is sliced away. The copied object has Entity's dynamic type, so e.attack() calls Entity::attack().

- With a base reference or pointer, no separate base object is copied. The original Dragon object's dynamic type remains available for virtual dispatch. The vptr is an implementation mechanism; do not assume a portable, literal vptr value or layout.

```
#include <iostream>

class Entity {
public:
    virtual ~Entity() = default;

    virtual void attack() const {
        std::cout << "Entity performs generic scratch."
                  << std::endl;
    }
};

class Dragon : public Entity {
public:
    void attack() const override {
        std::cout << "Dragon breathes scorching fire!"
                  << std::endl;
    }
};

void triggerEncounter(const Entity& e) {
    e.attack();
}

int main() {
    Dragon boss;
    triggerEncounter(boss);
    return 0;
}
```

```
Dragon breathes scorching fire!
```

SECTION B – PSEUDO-CODE WRITING QUESTIONS

Question 1: Universal Payment Gateway with Runtime Polymorphism

- PaymentGateway is an abstract interface. A PaymentGateway* can point to either concrete gateway; virtual dispatch selects the implementation based on the actual object.

```
ABSTRACT CLASS PaymentGateway
    PUBLIC:
        PURE VIRTUAL METHOD Authorize(amount AS REAL)
            RETURNS BOOLEAN
        PURE VIRTUAL METHOD Capture() RETURNS BOOLEAN
        VIRTUAL DESTRUCTOR()
        END
ENDCLASS

CLASS CreditCardGateway EXTENDS PaymentGateway
    PRIVATE:
        cardNumber: STRING
        cvv: STRING
        authorizedAmount: REAL

    PUBLIC:
        CONSTRUCTOR CreditCardGateway(card, securityCode)
            cardNumber <- card
            cvv <- securityCode
            authorizedAmount <- 0
        END

        OVERRIDE METHOD Authorize(amount AS REAL)
            IF LENGTH(cardNumber) = 16 AND LENGTH(cvv) = 3 THEN
                authorizedAmount <- amount
                OUTPUT "Card authorization approved"
                RETURN TRUE
            ELSE
                OUTPUT "Invalid card details"
                RETURN FALSE
            ENDIF
        END

        OVERRIDE METHOD Capture()
            IF authorizedAmount > 0 THEN
                OUTPUT "Credit card payment captured"
                RETURN TRUE
            ENDIF
            RETURN FALSE
        END
ENDCLASS

CLASS CryptoGateway EXTENDS PaymentGateway
    PRIVATE:
        walletHash: STRING
        gasFee: REAL
        authorizedAmount: REAL

    PUBLIC:
        CONSTRUCTOR CryptoGateway(wallet, fee)
            walletHash <- wallet
```

```
            gasFee <- fee
            authorizedAmount <- 0
        END

        OVERRIDE METHOD Authorize(amount AS REAL)
            IF STARTS_WITH(walletHash, "0x") AND gasFee <= amount THEN
                authorizedAmount <- amount
                OUTPUT "Crypto wallet verified"
                OUTPUT "Gas fee validation successful"
                RETURN TRUE
            ELSE
                OUTPUT "Crypto authorization failed"
                RETURN FALSE
            ENDIF
        END

        OVERRIDE METHOD Capture()
            IF authorizedAmount > 0 THEN
                OUTPUT "Cryptocurrency payment captured"
                RETURN TRUE
            ENDIF
            RETURN FALSE
        END
ENDCLASS

BEGIN Main
    DECLARE gatewayPtr AS POINTER TO PaymentGateway
    gatewayPtr <- NEW CryptoGateway("0xABCD1234", 15.00)

    IF gatewayPtr->Authorize(1500.00) = TRUE THEN
        CALL gatewayPtr->Capture()
    ENDIF

    DELETE gatewayPtr
END
```

Step	Operation / dispatch	Result
1	Base pointer receives CryptoGateway object	Runtime object is CryptoGateway
2	gatewayPtr->Authorize(1500.00)	CryptoGateway.Authorize executes
3	Wallet prefix check	0x prefix is valid
4	Gas fee check	15.00 ≤ 1500.00; valid
5	gatewayPtr->Capture()	CryptoGateway.Capture executes
6	DELETE gatewayPtr	Virtual destructor safely destroys derived object then base

```
Crypto wallet verified
Gas fee validation successful
Cryptocurrency payment captured
```

Question 2: Resilient Banking Engine with Nested Try-Catch and Rethrowing

- The intermediate wrapper catches the common TransactionException base, writes an audit record, and uses RETHROW to preserve the original derived exception. Main handles derived exception types before the base type.

```
CLASS TransactionException
    PUBLIC:
```

```
        reason: STRING
        CONSTRUCTOR TransactionException(message)
            reason <- message
        END
ENDCLASS

CLASS AccountFrozenException EXTENDS TransactionException
    CONSTRUCTOR AccountFrozenException(message)
        : BASE TransactionException(message)
    END
ENDCLASS

CLASS DailyLimitExceededException EXTENDS TransactionException
    PUBLIC:
        excessAmount: REAL
    CONSTRUCTOR DailyLimitExceededException(message, excess)
        : BASE TransactionException(message)
        excessAmount <- excess
    END
ENDCLASS

FUNCTION DebitLedger(amount AS REAL, isFrozen AS BOOLEAN)
    IF isFrozen = TRUE THEN
        THROW NEW AccountFrozenException("Account is frozen")
    ENDIF

    IF amount > 50000 THEN
        THROW NEW DailyLimitExceededException(
            "Daily withdrawal limit exceeded", amount - 50000)
    ENDIF

    OUTPUT "Debit successful"
END

FUNCTION AuditTransactionWrapper(amount, isFrozen)
    TRY
        CALL DebitLedger(amount, isFrozen)
    CATCH (e AS TransactionException)
        OUTPUT "SECURITY AUDIT: ", e.reason
        RETHROW
    ENDTRY
END

BEGIN Main
    TRY
        TRY
            CALL AuditTransactionWrapper(1000, TRUE)
        CATCH (e AS AccountFrozenException)
            OUTPUT "Transaction denied: account frozen."
        CATCH (e AS DailyLimitExceededException)
            OUTPUT "Excess amount: ", e.excessAmount
        ENDTRY
    CATCH (e AS TransactionException)
        OUTPUT "General transaction failure: ", e.reason
    ENDTRY
END
```

Sequence	Control flow / event	State or output

1	Main calls AuditTransactionWrapper(1000, TRUE)	Withdrawal amount = 1,000
2	Wrapper calls DebitLedger	Frozen flag is TRUE
3	DebitLedger throws AccountFrozenException	Normal flow stops
4	Wrapper catches TransactionException base	Derived exception binds to base reference
5	Wrapper writes audit entry	SECURITY AUDIT: Account is frozen
6	RETHROW executes	Original AccountFrozenException continues outward
7	Main's specific handler matches	Transaction denied: account frozen.
8	Function and handler scopes exit	No normal debit occurs

```
SECURITY AUDIT: Account is frozen
Transaction denied: account frozen.
```

Question 3: Polymorphic Game Engine with Virtual Destructors

- GameObject is abstract and declares pure virtual Update and Render methods plus a virtual destructor. ParticleSystem allocates a dynamic particle-coordinate array and releases it in its destructor.
- When DEALLOCATE is called through GameObject*, virtual destruction invokes ParticleSystem's destructor first, followed by GameObject's destructor.

```
ABSTRACT CLASS GameObject
    PUBLIC:
        PURE VIRTUAL METHOD Update(deltaTime AS REAL)
        PURE VIRTUAL METHOD Render()
        VIRTUAL DESTRUCTOR()
            OUTPUT "GameObject base destroyed"
        END
ENDCLASS

CLASS ParticleSystem EXTENDS GameObject
    PRIVATE:
        particleCoordinates AS POINTER TO REAL
        particleCount: INTEGER

    PUBLIC:
        CONSTRUCTOR ParticleSystem(count)
            particleCount <- count
            particleCoordinates <- NEW ARRAY[particleCount * 2] OF REAL
            OUTPUT "Particle memory allocated"
        END

        OVERRIDE METHOD Update(deltaTime AS REAL)
            OUTPUT "Particles updated"
        END

        OVERRIDE METHOD Render()
            OUTPUT "Particles rendered"
        END

        OVERRIDE DESTRUCTOR()
            DELETE ARRAY particleCoordinates
            OUTPUT "Particle memory released"
        END
```

```
ENDCLASS

BEGIN Main
    DECLARE sceneEntities AS ARRAY[1] OF POINTER TO GameObject
    sceneEntities[0] <- NEW ParticleSystem(500)

    FOR EACH entity IN sceneEntities
        CALL entity->Update(0.016)
        CALL entity->Render()
    ENDFOR

    FOR i <- 0 TO 0
        DEALLOCATE sceneEntities[i]
    ENDFOR
END
```

Order	Lifecycle event	Explanation
1	ParticleSystem constructor	Allocates particle coordinate array
2	Update and Render	Virtual calls dispatch to ParticleSystem
3	DEALLOCATE base pointer	Virtual destructor dispatch begins derived cleanup
4	ParticleSystem destructor	Releases particleCoordinates
5	GameObject destructor	Base subobject cleanup completes

```
Particle memory allocated
Particles updated
Particles rendered
Particle memory released
GameObject base destroyed
```

Question 4: File Stream Pipeline with Stack Unwinding and Catch-All

- ResourceGuard follows RAII: its constructor acquires the resource and its destructor releases it. When CRCValidationException is thrown, stack unwinding destroys local automatic objects before control enters the matching catch handler.

```
CLASS ResourceGuard
    PRIVATE:
        handle: INTEGER

    PUBLIC:
        CONSTRUCTOR ResourceGuard(fileName)
            handle <- OPEN_FILE(fileName)
            OUTPUT "Resource acquired"
        END

        DESTRUCTOR()
            CLOSE_FILE(handle)
            OUTPUT "Resource released"
        END
ENDCLASS

CLASS CRCValidationException
    PUBLIC:
        expectedChecksum: INTEGER
        calculatedChecksum: INTEGER
```

```
    CONSTRUCTOR CRCValidationException(expected, calculated)
        expectedChecksum <- expected
        calculatedChecksum <- calculated
    END
ENDCLASS

FUNCTION IngestDataStream(fileName)
    DECLARE guard AS ResourceGuard(fileName)
    READ data blocks FROM guard.handle
    DECLARE expected AS INTEGER <- READ_EXPECTED_CRC()
    DECLARE calculated AS INTEGER <- COMPUTE_CRC()

    IF expected != calculated THEN
        THROW NEW CRCValidationException(expected, calculated)
    ENDIF

    OUTPUT "File validation successful"
END

BEGIN Main
    TRY
        CALL IngestDataStream("telemetry.bin")
    CATCH (e AS CRCValidationException)
        OUTPUT "CRC mismatch. Expected: ", e.expectedChecksum
        OUTPUT "Calculated: ", e.calculatedChecksum
    CATCH (...)
        OUTPUT "Unexpected ingestion failure"
    ENDTRY
END
```

Step	Event	Control-flow result
1	Main enters TRY and calls IngestDataStream	Function begins
2	ResourceGuard constructor runs	File handle acquired
3	Data is read and CRC is calculated	Checksums differ
4	CRCValidationException is thrown	Function's normal path is abandoned
5	Stack unwinding begins	Local ResourceGuard destructor runs
6	ResourceGuard destructor closes handle	Resource released before catch
7	Main's specific CATCH executes	Displays expected and calculated CRC
8	CATCH (...)	Not executed because specific handler matched

```
Resource acquired
Resource released
CRC mismatch. Expected: 1234
Calculated: 5678
```

The checksum values 1234 and 5678 are illustrative trace values because the question bank does not provide actual file contents or checksum values.

Question 5: Scientific Telemetry Calibration with noexcept Contracts

- CalibrateThermalSensors is potentially throwing (noexcept(FALSE)); it throws OutOfBoundsException if the computed temperature is outside −273.15°C to +1500.0°C.
- CommitTelemetry is declared NOEXCEPT. It must catch and suppress all exceptions internally; allowing an exception to escape a noexcept function causes std::terminate in C++.

```
CLASS OutOfBoundsException
    PUBLIC:
        measuredTemperature: REAL
    CONSTRUCTOR OutOfBoundsException(temp)
        measuredTemperature <- temp
    END
ENDCLASS

FUNCTION CalibrateThermalSensors(rawVoltage AS REAL)
    RETURNS REAL NOEXCEPT(FALSE)
    DECLARE temperature AS REAL
    temperature <- CONVERT_VOLTAGE_TO_TEMPERATURE(rawVoltage)

    IF temperature < -273.15 OR temperature > 1500.0 THEN
        THROW NEW OutOfBoundsException(temperature)
    ENDIF

    RETURN temperature
END

FUNCTION CommitTelemetry(finalTemp AS REAL) NOEXCEPT
    TRY
        CALL WRITE_TO_AVIONICS_BUS(finalTemp)
        OUTPUT "Telemetry committed"
    CATCH (...)
        OUTPUT "Commit failure suppressed by NOEXCEPT handler"
    ENDTRY
END

BEGIN Main
    TRY
        DECLARE rawVoltage AS REAL <- INPUT_VOLTAGE()
        DECLARE finalTemp AS REAL
        finalTemp <- CalibrateThermalSensors(rawVoltage)
        CALL CommitTelemetry(finalTemp)
    CATCH (e AS OutOfBoundsException)
        OUTPUT "Calibration rejected. Temperature: ",
               e.measuredTemperature, " C"
    CATCH (...)
        OUTPUT "Unexpected telemetry pipeline error"
    ENDTRY
END
```

Step	Operation	Result
1	Main supplies a faulty sensor voltage	Conversion produces −300.0°C
2	CalibrateThermalSensors validates bounds	−300.0 < −273.15
3	OutOfBoundsException is thrown	Calibration function exits exceptionally
4	CommitTelemetry is not called	No invalid reading is committed
5	Main catches OutOfBoundsException	Specific handler displays −300.0°C
6	Catch-all handler	Not executed

```
Calibration rejected. Temperature: -300 C
```

The voltage-to-temperature conversion formula is not specified in the question bank. The trace therefore uses the stated faulty reading of −300.0°C as the computed result, rather than inventing a conversion formula or voltage value.